SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEIGN AND CONNECTIVITY

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 1 – DEIGN AND CONNECTIVITY
Weightage:		**Total Marks:**	40 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:		**Reg. No.:**	
Section / Batch:		**Date of Submission:**	
Faculty In-charge:	**Dr. B.SHYAMALA GOWRI**	**Project Mode:**	Individual Submission

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

Requirement Analysis (5 Marks)	ER Diagram Design (10 Marks)	Table Design & Normalization (5 Marks)	Database Connectivity using Python (20 Marks)	CRUD Operations (5 Marks)	Total (35 Marks)

Scenario 4: Library Management System

A large academic library manages thousands of books, journals, magazines, and digital resources. Students and faculty members borrow and return books regularly. The current manual system often results in misplaced records and inaccurate tracking of issued books. The library wants to automate its operations by maintaining information about books, authors, publishers, members, and transactions. A member may borrow multiple books, and a book may be issued to different members over time. The system should also calculate fines for overdue books and generate inventory reports. Design an efficient database schema and implement database connectivity using Python to manage book records and member transactions.

1. Requirement Analysis (5 Marks)

1.1 Introduction

A Library Management System is a database application designed to automate library operations such as maintaining book records, member registration, book issue and return, and fine calculation. Instead of maintaining records manually, the system stores all information in a centralized relational database, enabling efficient management, faster searches, and accurate reporting.

The system uses SQL databases (SQLite, MySQL, or PostgreSQL) with Python for database connectivity and supports CRUD (Create, Read, Update, Delete) operations.

1.2 Existing System

The current library system is managed manually using registers or spreadsheets.

Limitations

- Manual entry consumes more time.
- Book records may be duplicated.
- Difficult to search for books.
- Fine calculation is manual.
- Report generation is difficult.

- Records may be lost or damaged.
- Updating information is cumbersome.

1.3 Proposed System

The proposed Library Management System is a computerized solution that stores all library information in a relational database.
The system enables librarians to:

- Add new books.
- Register members.
- Issue books.
- Return books.
- Calculate fines.
- Search books quickly.
- Generate reports.

Python connects with SQL databases to perform CRUD operations efficiently.

1.4 Objectives

- Develop a centralized database.
- Reduce manual work.
- Store accurate book information.
- Maintain member records.
- Track issued books.
- Calculate overdue fines.
- Generate reports.
- Perform CRUD operations using Python.

1.5 Scope

The system can be used in:

- Schools

- Colleges
- Universities
- Public libraries
- Digital libraries

1.6 Functional Requirements

The system should:

- Add books
- Add members
- Search books
- Issue books
- Return books
- Calculate fines
- Update records
- Delete records
- Display reports

1.7 Non-Functional Requirements

- Easy to use
- Secure
- Fast processing
- Reliable
- Scalable
- Maintainable

2. ER Diagram Design (10 Marks)

Entities

1. **Member**
2. **Book**
3. **Author**
4. **Publisher**
5. **Issue**
6. **Fine**

Attributes
MEMBER

- **Member_ID (PK)**
- **Member_Name**
- **Gender**
- **Phone**
- **Email**
- **Address**

BOOK

- **Book_ID (PK)**
- **Title**
- **Category**
- **Price**
- **Author_ID (FK)**
- **Publisher_ID (FK)**

AUTHOR

- **Author_ID (PK)**
- **Author_Name**

PUBLISHER

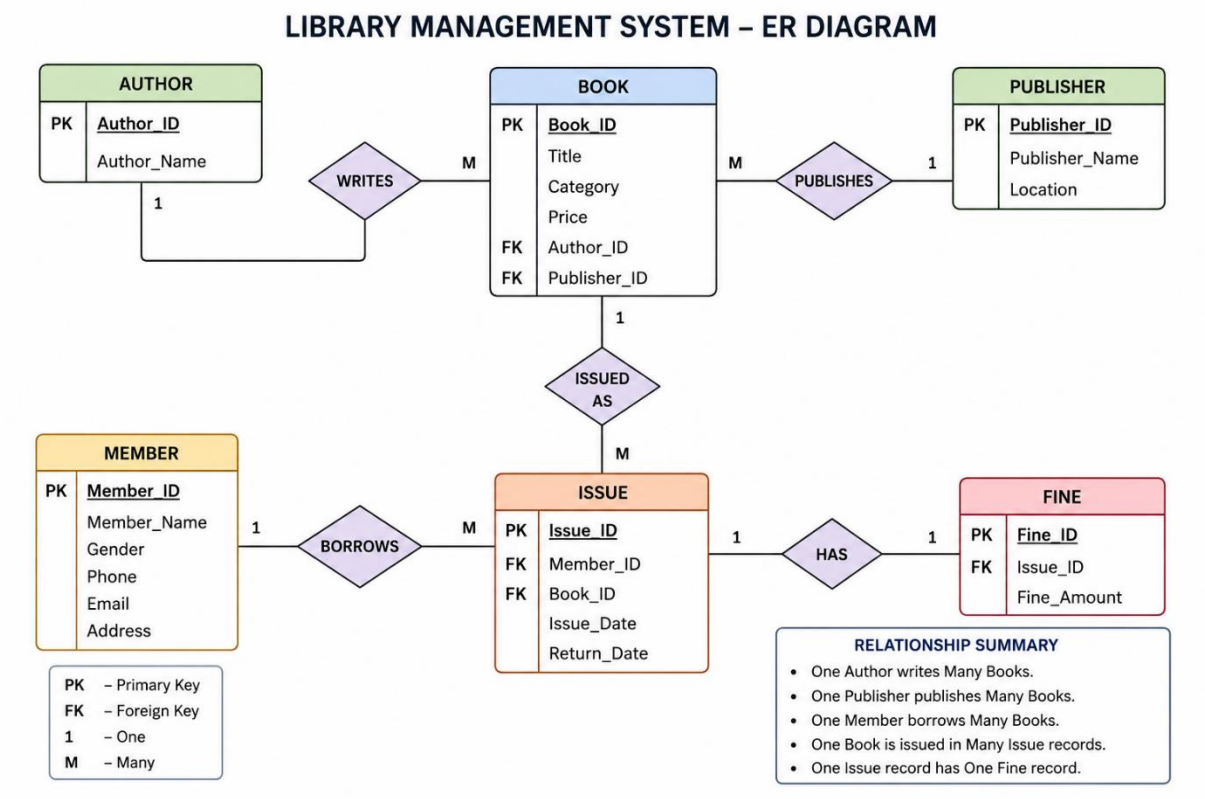
- **Publisher_ID (PK)**
- **Publisher_Name**
- **Location**

ISSUE

- **Issue_ID (PK)**
- **Member_ID (FK)**
- **Book_ID (FK)**
- **Issue_Date**
- **Return_Date**

FINE

- **Fine_ID (PK)**
- **Issue_ID (FK)**
- **Fine_Amount**



Entity 1	Relationship	Entity 2
Author	1 : M	Book
Publisher	1 : M	Book
Member	1 : M	Issue
Book	1 : M	Issue
Issue	1 : 1	Fine

3. Table Design & Normalization (5 Marks)

Column	Data Type	Constraint
Member_ID	INT	Primary Key
Member_Name	VARCHAR(100)	NOT NULL
Gender	VARCHAR(10)	
Phone	VARCHAR(15)	

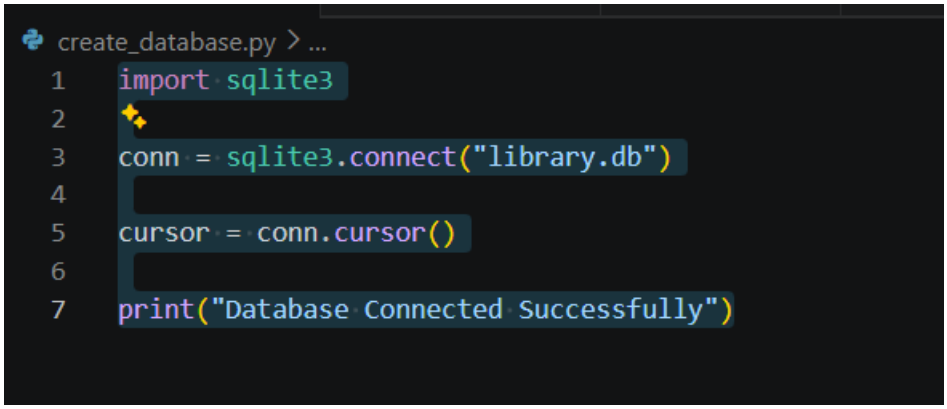
Email	**VARCHAR(100)**	**UNIQUE**
Address	**VARCHAR(200)**	

Column	**Data Type**	**Constraint**
Author_ID	**INT**	**Primary Key**
Author_Name	**VARCHAR(100)**	**NOT NULL**

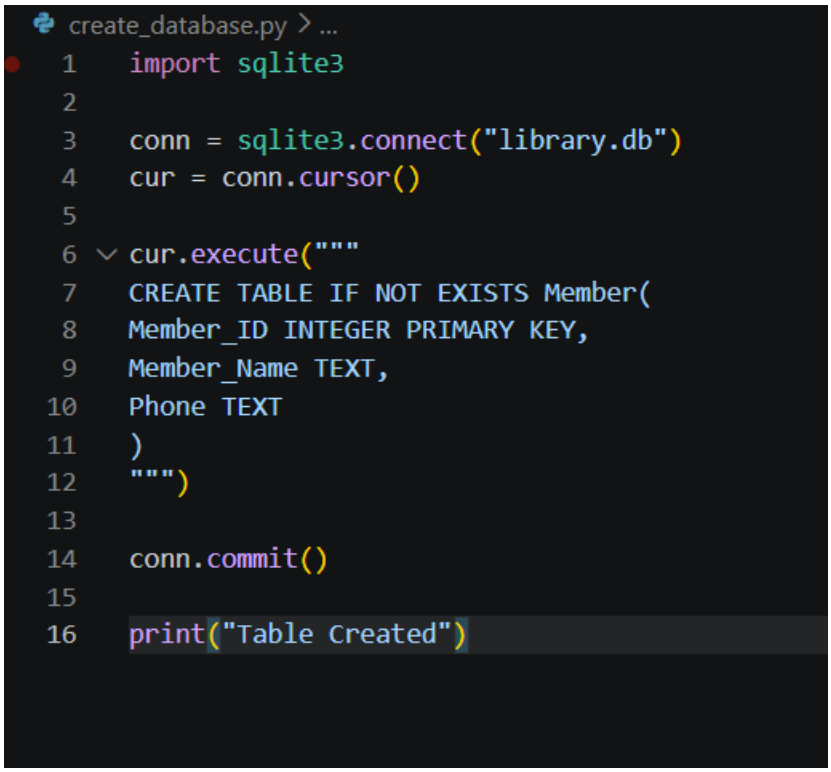
Column	**Data Type**	**Constraint**
Publisher_ID	**INT**	**Primary Key**
Publisher_Name	**VARCHAR(100)**	**NOT NULL**
Location	**VARCHAR(100)**	

Column	**Data Type**	**Constraint**
Book_ID	**INT**	**Primary Key**
Title	**VARCHAR(100)**	**NOT NULL**
Category	**VARCHAR(50)**	
Price	**DECIMAL(8,2)**	
Author_ID	**INT**	**Foreign Key**
Publisher_ID		**Foreign Key**

7. Python Database Connectivity (SQLite)



8. Create Tables Using Python



```python
import sqlite3

# Create new database
conn = sqlite3.connect("library.db")
cur = conn.cursor()

# Create table
cur.execute("""
CREATE TABLE Member(
    Member_ID INTEGER PRIMARY KEY,
    Member_Name TEXT,
    Gender TEXT,
    Phone TEXT,
    Email TEXT,
    Address TEXT
)
""")

# Insert records
cur.execute("INSERT INTO Member VALUES (101,'Asha','Female','9876543210','asha@gmail.com','Chennai')")
cur.execute("INSERT INTO Member VALUES (102,'Karan','Male','9123456789','karan@gmail.com','Bangalore')")
cur.execute("INSERT INTO Member VALUES (103,'Priya','Female','9988776655','priya@gmail.com','Hyderabad')")

conn.commit()

print("Records Inserted Successfully")

conn.close()
```

```
OneDrive/Desktop/library/display.py
(101, 'Asha', 'Female', '9999999999', 'asha@gmail.com', 'Chennai')
(102, 'Karan', 'Male', '9123456789', 'karan@gmail.com', 'Bangalore')
(103, 'Priya', 'Female', '9988776655', 'priya@gmail.com', 'Hyderabad')
PS C:\Users\linga\OneDrive\Desktop\library>
```

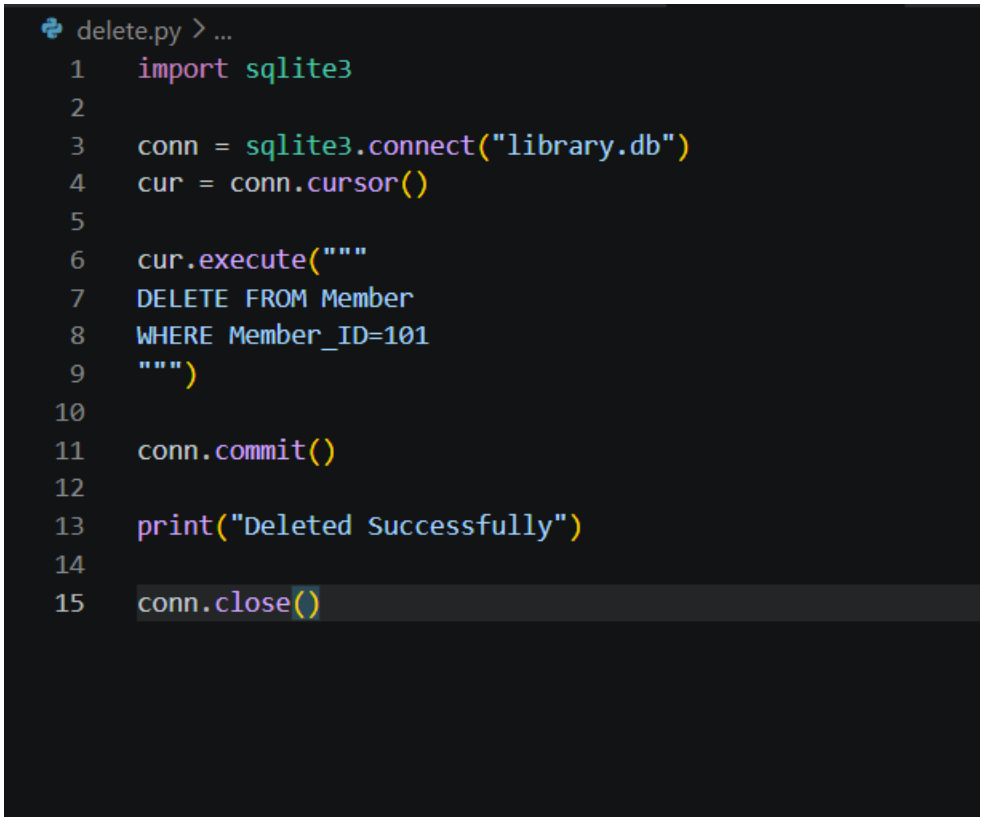
```python
import sqlite3

conn = sqlite3.connect("library.db")
cur = conn.cursor()

cur.execute("SELECT * FROM Member")

rows = cur.fetchall()

for row in rows:
    print(row)

conn.close()
```

```
Member ID : 101
Name      : Asha
Gender    : Female
Phone     : 9999999999
Email     : asha@gmail.com
Address   : Chennai
----------------------------------------
Member ID : 102
Name      : Karan
Gender    : Male
```

update.py > ...

```python
import sqlite3

conn = sqlite3.connect("library.db")
cur = conn.cursor()

cur.execute("""
UPDATE Member
SET Phone='9999999999'
WHERE Member_ID=101
""")

conn.commit()

print("Record Updated Successfully")

conn.close()
```

```
OneDrive/Desktop/library/update.py
Record Updated Successfully
PS C:\Users\linga\OneDrive\Desktop\library> & C:\Users\linga\AppData\Local\Programs\Python\Python312\python.exe c:/Users/linga/
OneDrive/Desktop/library/display.py
(101, 'Asha', 'Female', '9999999999', 'asha@gmail.com', 'Chennai')
(102, 'Karan', 'Male', '9123456789', 'karan@gmail.com', 'Bangalore')
(103, 'Priya', 'Female', '9988776655', 'priya@gmail.com', 'Hyderabad')
PS C:\Users\linga\OneDrive\Desktop\library>
```



Scenario 9: Online Learning Management System

An educational technology company provides online courses to students worldwide. Students enroll in courses, watch video lectures, submit assignments, and receive grades. Instructors create courses and manage learning materials. The platform needs a database system to maintain student profiles, instructor details, course catalogs, enrollments, assignments, and assessment results. Administrators require reports on student progress, course completion rates, and instructor performance. Design a relational database schema that supports these requirements and implement Python programs for managing enrollments and academic records.

1. Requirement Analysis (5 Marks)

Objective

Develop a relational database for an Online Learning Management System that stores and manages:

- Student information
- Instructor information
- Courses
- Student enrollments
- Assignments
- Grades

The system should allow administrators to

- Register students
- Add instructors
- Create courses
- Enroll students
- Submit grades
- View reports

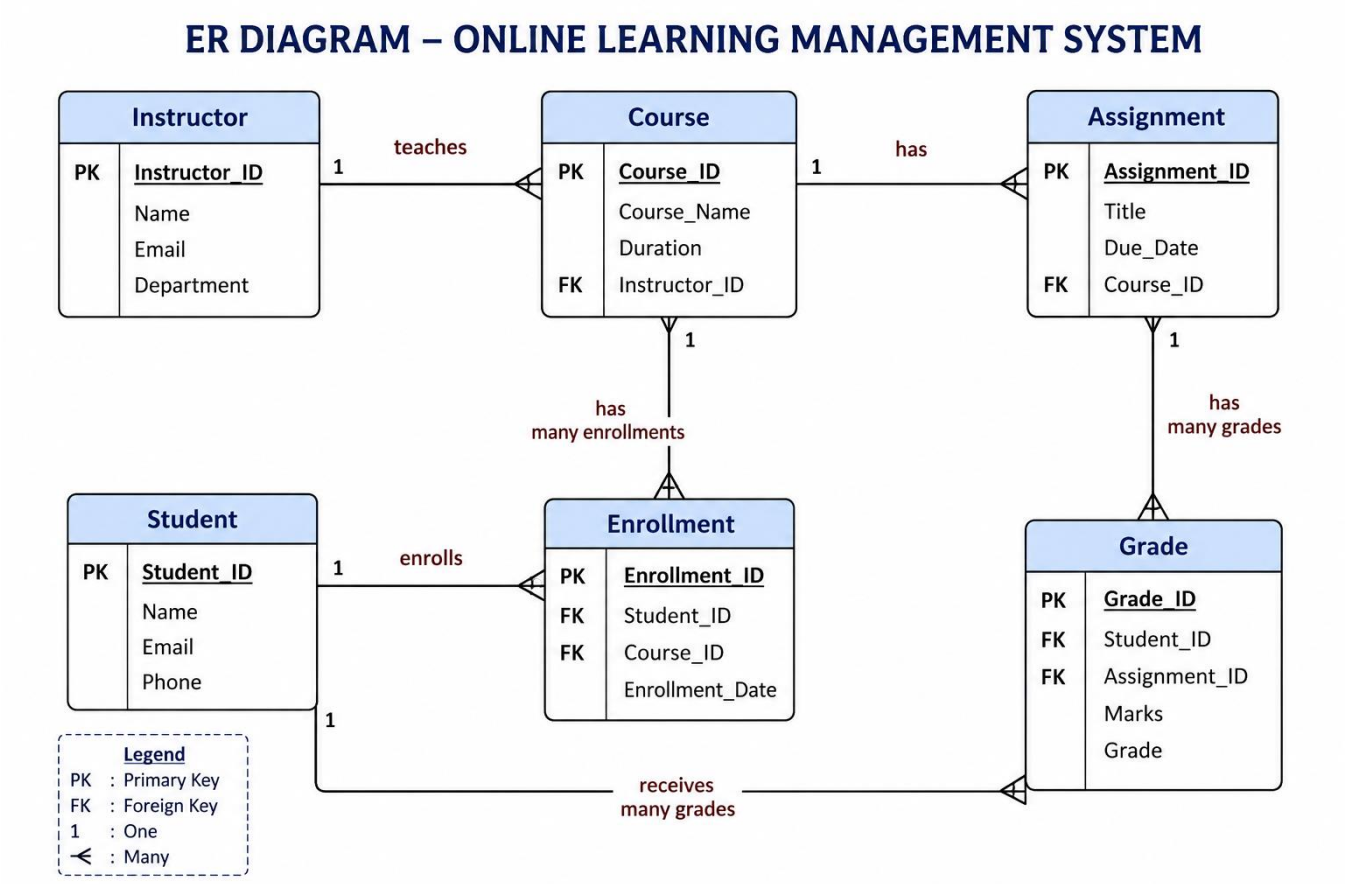
Functional Requirements

- Student Registration
- Instructor Registration
- Course Creation
- Student Enrollment
- Assignment Creation
- Grade Management
- Student Progress Report
- CRUD Operations

Non-Functional Requirements

- Data Integrity
- Security
- Fast Retrieval
- Scalability
- Easy Maintenance

2. ER Diagram



3. Tables

Student

Field	Type
Student_ID	INT PK
Name	VARCHAR(50)
Email	VARCHAR(50)
Phone	VARCHAR(15)

Field	Type
Instructor_ID	INT PK
Name	VARCHAR(50)
Email	VARCHAR(50)
Department	VARCHAR(40)

Field	Type
Course_ID	INT PK
Course_Name	VARCHAR(50)
Duration	VARCHAR(20)
Instructor_ID	INT FK

Field	Type

Field	Type
Enrollment_ID	INT PK
Student_ID	INT FK
Course_ID	INT FK
Enrollment_Date	DATE

Field	Type
Assignment_ID	INT PK
Title	VARCHAR(50)
Due_Date	DATE
Course_ID	INT FK

Field	Type
Grade_ID	INT PK
Student_ID	INT FK
Assignment_ID	INT FK
Marks	INT
Grade	CHAR(2)

4. Normalization (5 Marks)

1NF

- **No repeating groups**
- **Atomic values**

2NF

- **Every non-key attribute depends on the primary key**

3NF

- **No transitive dependency**
- **Foreign keys maintain relationships**

```
import psycopg2

# ------------------ DATABASE CONNECTION ------------------ #

try:
    conn = psycopg2.connect(
        host="localhost",
        database="lms",
        user="postgres",
        password="56789",      # Your PostgreSQL password
        port="5432"
    )

    cur = conn.cursor()
```

```
    print("Connected Successfully!")

except Exception as e:
    print("Connection Error:", e)
    exit()

# ------------------ CREATE TABLE ------------------ #

cur.execute("""
CREATE TABLE IF NOT EXISTS Student(
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(100),
    Phone VARCHAR(15)
)
""")

conn.commit()

print("Student table created successfully!")

# ------------------ INSERT ------------------ #

try:

    student_id = int(input("Enter Student ID : "))
    name = input("Enter Name : ")
    email = input("Enter Email : ")
    phone = input("Enter Phone : ")

    cur.execute("""
    INSERT INTO Student(Student_ID, Name, Email, Phone)
    VALUES(%s,%s,%s,%s)
    """, (student_id, name, email, phone))

    conn.commit()

    print("Student inserted successfully!")

except Exception as e:

    conn.rollback()

    print("Insert Error:", e)

# ------------------ DISPLAY ------------------ #
```

```
try:

    cur.execute("SELECT * FROM Student")

    rows = cur.fetchall()

    print("\nStudent Details")

    for row in rows:
        print(row)

except Exception as e:

    conn.rollback()

    print("Display Error:", e)

# ------------------ UPDATE ------------------ #

try:

    student_id = int(input("\nEnter Student ID to update : "))
    new_phone = input("Enter New Phone : ")

    cur.execute("""
    UPDATE Student
    SET Phone=%s
    WHERE Student_ID=%s
    """, (new_phone, student_id))

    conn.commit()

    print("Record Updated Successfully!")

except Exception as e:

    conn.rollback()

    print("Update Error:", e)

# ------------------ DISPLAY AFTER UPDATE ------------------ #

cur.execute("SELECT * FROM Student")

rows = cur.fetchall()

print("\nStudent Details After Update")
```

```
for row in rows:
    print(row)

# ------------------ DELETE ------------------ #

try:

    student_id = int(input("\nEnter Student ID to delete : "))

    cur.execute("""
    DELETE FROM Student
    WHERE Student_ID=%s
    """, (student_id,))

    conn.commit()

    print("Record Deleted Successfully!")

except Exception as e:

    conn.rollback()

    print("Delete Error:", e)

# ------------------ DISPLAY AFTER DELETE ------------------ #

cur.execute("SELECT * FROM Student")

rows = cur.fetchall()

print("\nStudent Details After Delete")

for row in rows:
    print(row)

# ------------------ CLOSE CONNECTION ------------------ #

cur.close()
conn.close()

print("\nConnection Closed.")
```

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Us
Connected Successfully!
Student table created successfully!
Enter Student ID : 2
Enter Name : Rahul
Enter Email : rahul@gmail.com
Enter Phone : 9876543211
Student inserted successfully!

Student Details
(1, 'Sania', 'sania@gmail.com', '9876543210')
(2, 'Rahul', 'rahul@gmail.com', '9876543211')

Enter Student ID to update : 2
Enter New Phone :  9999999999
Record Updated Successfully!

Student Details After Update
(1, 'Sania', 'sania@gmail.com', '9876543210')
(2, 'Rahul', 'rahul@gmail.com', ' 9999999999')

Enter Student ID to delete : 2
Record Deleted Successfully!

Student Details After Delete
(1, 'Sania', 'sania@gmail.com', '9876543210')

Connection Closed.
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```